

NeuralForge

Complete Platform Guide

Self-hosted AI Command Center

12

Services

84

APIs

6

Tabs

24

MCP Tools

30+

Models

localhost:9000

Version 1.0 | March 2026

Table of Contents

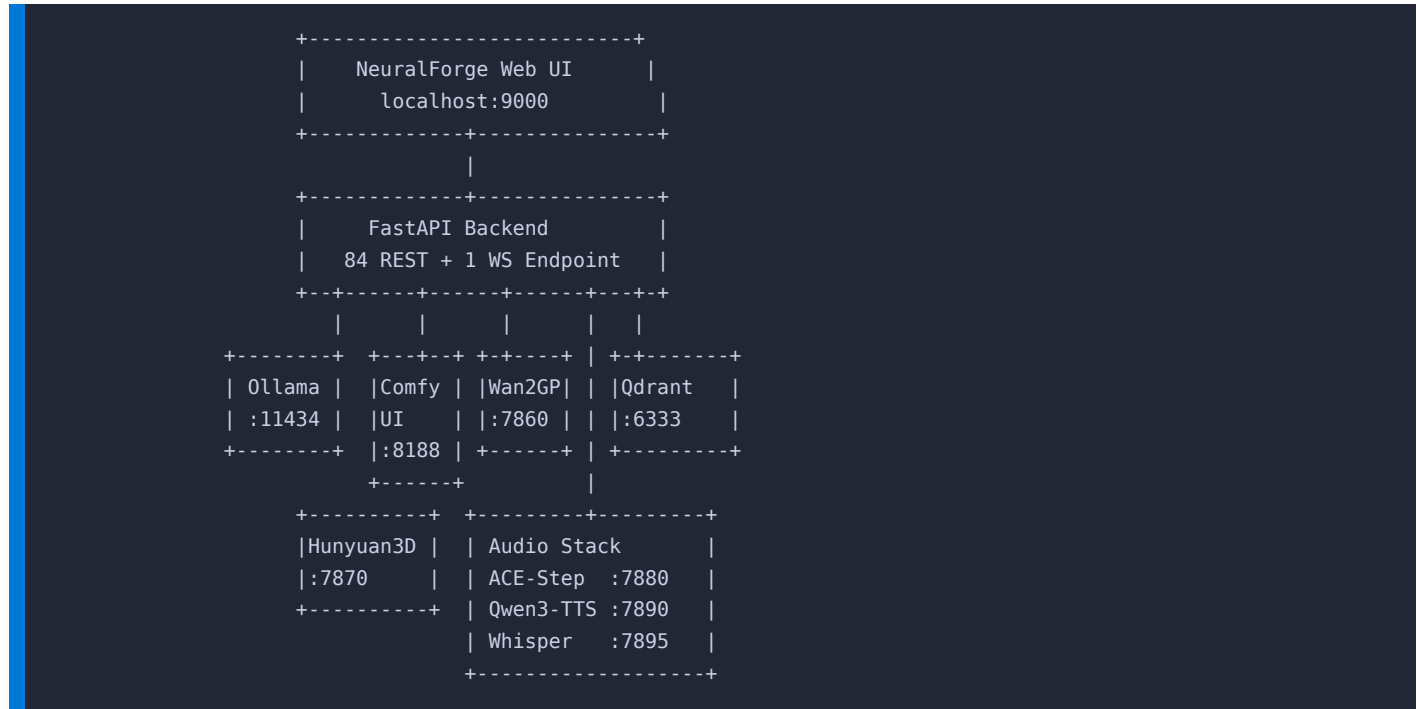
1.	Overview & Architecture	3
2.	Dashboard	4
3.	Module System	5
4.	AI Model Stack	6
5.	AI Agents	8
6.	RAG (Retrieval-Augmented Generation)	10
7.	Telegram AI Bot	12
8.	LoRA Fine-Tuning	14
9.	Generation Pipeline	16
10.	SMM AI Department	17
11.	MCP Server	19
12.	Installation & Startup	21
13.	API Reference (84 Endpoints)	23
14.	Hardware Requirements	26

1. Overview & Architecture

NeuralForge is a self-hosted AI command center accessible at localhost:9000. It consolidates 12 services behind a unified web interface, exposing 84 API endpoints for full programmatic control. The platform is organized into 6 primary tabs: Dashboard, Agents, RAG, Telegram, LoRA, and SMM.

It runs entirely on your hardware with no cloud dependencies, giving you complete control over your AI infrastructure, data privacy, and operational costs. Every component communicates over localhost, and the web UI provides real-time monitoring of all running services.

Architecture Diagram



Core Principles

- Fully local: all processing on your GPU/CPU, zero cloud API calls required
- Unified control: single web interface to manage all 11 AI services
- Smart VRAM management: exclusive GPU groups prevent out-of-memory crashes
- Extensible: YAML module system, full REST API, MCP integration for Claude Code
- Resilient: systemd-managed, auto-restart on failure, health monitoring

Six Platform Tabs

- Dashboard:** Real-time hardware monitoring, service lifecycle, health alerts, GPU/RAM/Disk metrics
- Agents:** 13 AI roles with 9 tools, solo/team/orchestrator modes, file upload, history
- RAG:** Document indexing with ONNX GPU embeddings (1800 docs/s), multi-collection semantic search
- Telegram:** AI auto-responder from your account, 14 personas, voice clone, vision analysis
- LoRA:** Fine-tune LLMs with Unsloth, 16 base models, live training monitor, JSONL datasets
- SMM:** Social media management: trend scanning, post generation, 7-platform publishing, analytics

2. Dashboard

The Dashboard is the operational nerve center of NeuralForge. It provides real-time hardware telemetry, service lifecycle management, and health alerting at a glance. All data refreshes automatically via WebSocket connection.

Live Metrics

- GPU: VRAM usage, temperature, fan speed, power draw (nvidia-smi real-time)
- RAM: total / used / available / swap usage
- CPU: per-core utilization, load averages
- Disk: mount points, free space, usage percentage per partition

Service Management

Each of the 11 modules can be started and stopped individually from the UI. Services marked as heavy_gpu belong to an exclusive GPU group — starting one automatically stops others in the same group to prevent VRAM exhaustion. Service types include systemd units, direct processes, and Docker containers.

Monitoring Widgets

- Ollama Models — currently loaded models, parameter count, quantization level
- GPU Processes — per-process VRAM consumption with PID and command
- Qdrant Collections — vector count, embedding dimension, health status
- Storage Overview — disk usage breakdown by category (models, logs, data, cache)

Health Alert Thresholds

Metric	Critical When	Level	Warning When	Level
GPU Temperature	> 85°C	CRITICAL	> 75°C	WARNING
VRAM Usage	> 95%	CRITICAL	> 85%	WARNING
RAM Available	< 2 GB	CRITICAL	< 5 GB	WARNING
Disk Free	< 10 GB	CRITICAL	< 50 GB	WARNING

Quick Actions

- Start Basics:** Launch Ollama + Qdrant + Open WebUI in one click
- Stop Heavy:** Shut down all heavy_gpu services to free VRAM instantly
- Free VRAM:** Unload all Ollama models from GPU memory without stopping the service

API Keys Management

API keys and tokens are stored in secrets.json and managed through the Dashboard UI. Keys are masked in the interface and can be added, edited, or removed without touching config files. Supported keys include: Telegram API hash, Twitter OAuth tokens, LinkedIn, Facebook, Instagram, Threads, and Bluesky credentials.

3. Module System

NeuralForge uses a YAML-based module system for declaring, configuring, and managing services. Each module specifies its type (systemd, process, or docker), port, VRAM requirements, health check URL, and GPU group membership.

Module Registry (11 Services)

Service	Type	Port	VRAM	Group	Description
Ollama	systemd	11434	—	—	LLM inference engine (30+ models)
ComfyUI	process	8188	8-22 GB	heavy_gpu	Image generation (FLUX Klein)
Wan2GP	process	7860	12-24 GB	heavy_gpu	Video generation (Wan 2.2)
Hunyuan3D	process	7870	13-20 GB	heavy_gpu	3D model generation
ACE-Step	process	7880	4-6 GB	—	AI music generation
Qwen3-TTS	process	7890	4-5 GB	—	Text-to-speech / voice clone
Whisper	process	7895	2-10 GB	—	Speech-to-text (99 languages)
Qdrant	docker	6333	—	—	Vector database for RAG
Open WebUI	docker	8080	—	—	Chat interface for Ollama
Perplexica	docker	3000	—	—	AI-powered web search
SearXNG	docker	8888	—	—	Meta search engine

Services in the same heavy_gpu group are mutually exclusive. Starting ComfyUI will auto-stop Wan2GP and Hunyuan3D to free VRAM.

Example Module YAML

```
comfyui:
  name: "ComfyUI"
  type: process
  port: 8188
  vram_min: 8
  vram_max: 22
  group: heavy_gpu
  start_cmd: "python main.py --listen 0.0.0.0 --port 8188"
  workdir: "/opt/ComfyUI"
  health_check: "http://localhost:8188/system_stats"
```

Module Lifecycle

Modules transition through states: stopped -> starting -> running -> stopping -> stopped. Health checks poll the configured URL every 5 seconds. If a process crashes, the status automatically updates. Logs are captured to /tmp/ai-panel-logs/{module}.log and the last 50 lines are available via the API.

4. AI Model Stack

NeuralForge manages 30+ AI models across six categories. All LLMs are served through Ollama, while specialized models (vision, audio, generation) run as dedicated services.

Large Language Models (via Ollama)

Model	VRAM	Speed	Use Case
Qwen 3.6 35B-A3B	23 GB	fast (MoE)	Default workhorse: agents, writer, critic
Qwen 3.8 27B	17 GB	~30 tok/s	Dense, high quality; tutor, vision fallback
Qwen 3.6 27B	17 GB	~30 tok/s	SMM trend scan + post writing
Gemma 4 26B	17 GB	~30 tok/s	Strongest multilingual (translator)
Qwen3-Coder 30B	18 GB	fast (MoE)	Coder, tester, security auditor
Nemotron 3 Nano 30B	24 GB	~25 tok/s	NVIDIA, long context, RAG answers
Mistral Small 24B	14 GB	~35 tok/s	Summarizer, balanced quality/speed
DeepSeek-R1 32B	19 GB	~22 tok/s	Chain-of-thought: analyst, trade analyst
DeepSeek-R1 14B	9 GB	~45 tok/s	Lightweight reasoning
Phi-4 Reasoning 14B	11 GB	~40 tok/s	Microsoft, math & logic specialist
Phi 4 14B	9 GB	~42 tok/s	Compact general purpose
Qwen 3.5 9B	6.6 GB	~60 tok/s	Telegram bot, quick SMM generation

Vision Models

Model	VRAM	Use Case
MiniCPM-V 8B	5.5 GB	Telegram bot photo analysis, auto VRAM swap
Qwen3-VL 8B	5.5 GB	Agent tool analyze_image, video understanding

Embedding Models

Model	VRAM	Throughput	Notes
bge-m3 (ONNX GPU)	~1 GB	1800 docs/s	RAG indexing, 180x faster than Ollama
bge-m3 (Ollama)	1.2 GB	~10 docs/s	Query embeddings / fallback
nomic-embed-text	0.3 GB	-	Optional small embedding model

Audio Models

Model	VRAM	Description
Whisper (large-v3)	2-10 GB	Speech-to-text, 99 languages
Qwen3-TTS	4-5 GB	Text-to-speech, voice clone from 3s sample
ACE-Step	4-6 GB	AI music generation from text prompts

Generation Models

Model	VRAM	Engine	Notes
FLUX Klein	8-22 GB	ComfyUI	Fast image gen, SDXL-quality
Wan 2.2	12-24 GB	Wan2GP	Video from text/image prompts

Hunyuan3D v2	13-20 GB	Service	3D model generation
--------------	----------	---------	---------------------

5. AI Agents

The Agent system provides specialized AI roles that can work solo, in sequential teams, or under an AI orchestrator. Each agent has a defined persona, configurable tool access, and produces structured output. Agents are powered by the universal.py engine and support file uploads, PDF generation, and persistent task history.

13 Agent Roles

Role	Description	Default Model
Researcher	Deep web research, source validation	qwen3.6:35b-a3b
Programmer	Code generation, review, debugging	qwen3-coder:30b
Data Analyst	CSV/JSON analysis, visualization	deepseek-r1:32b
Content Manager	Writes texts, articles, blog posts	qwen3.6:35b-a3b
Summarizer	Text condensation, key point extract	mistral-small:24b
Critic-Editor	Fact checking, quality improvement	qwen3.6:35b-a3b
Translator	RU, EN, ET, DE, FR, ES + 5 more	gemma4:26b
Email Assistant	Professional email drafting	qwen3.6:35b-a3b
Tester	Test case generation, bug finding	qwen3-coder:30b
Trade Analyst	Market and financial analysis	deepseek-r1:32b
Tutor	Educational step-by-step guides	qwen3.8:27b
Security Auditor	Vulnerability assessment, code audit	qwen3-coder:30b
Custom Agent	Full customization of role + tools	user choice

9 Agent Tools

Tool ID	Description
web_search	Search the web via SearXNG / Perplexica
read_url	Fetch and parse any web page content
run_python	Execute Python code in sandboxed environment
read_file	Read local files on the server
write_file	Write / create files on the server
analyze_file	Deep analysis of documents (CSV, JSON, etc.)
analyze_image	Vision model image understanding
rag_search	Search RAG knowledge base collections
deep_scrape	Multi-page website scraping with depth control

3 Execution Modes

- Solo:** Single agent runs with access to all enabled tools. Good for focused tasks.
- Team:** Sequential chain — each agent receives the prior agent's output as context. Example: Researcher -> Writer -> Critic produces researched, polished content.
- Orchestrator:** AI meta-agent auto-selects agents and tools based on the task. Retries if quality score < 7/10. Fully autonomous multi-step workflows.

15 Team Presets

- Research & Write
- Market Analysis

- Code Review Pipeline
- Full Article Workflow
- Data Report
- Security Audit
- Translation Chain
- Email Draft & Review
- Test Generation
- Tutorial Creation
- Code + Test + Review
- Summarize & Translate
- Content + SEO
- Deep Research
- Custom Chain

6. RAG (Retrieval-Augmented Generation)

The RAG module enables knowledge-augmented conversations by indexing your documents into a Qdrant vector database and retrieving relevant context at query time. It uses ONNX-accelerated GPU embeddings for indexing throughput that makes large-scale document processing practical.

Embedding Performance Comparison

Engine	Throughput	VRAM	Notes
bge-m3 ONNX GPU	1,800 docs/sec	~1 GB	Default, production-grade
Ollama bge-m3	10 docs/sec	~1.2 GB	Fallback, same vectors as ONNX
CPU-only bge-m3	50 docs/sec	0 GPU	Fallback when GPU busy

ONNX GPU embedding is 180x faster than Ollama-based embedding. Indexing 10,000 documents takes ~5.5 seconds vs ~17 minutes.

Supported Document Formats

Format	Extension	Processing
PDF	.pdf	Text extraction with page number tracking
Text	.txt	Plain text, line-based chunking
Markdown	.md	Markdown with header-aware splitting
Word	.docx	Microsoft Word document extraction
CSV	.csv	Tabular data, row-level chunking
HTML	.html	Web page content extraction, tag stripping

Key Features

- Multi-collection search — query across multiple knowledge bases simultaneously
- Chat with citations — responses include source document references with page numbers
- Upload & Index — drag-and-drop file upload with instant indexing
- Directory indexing — recursively index entire folders
- Collection management — create, browse, and delete collections from the UI
- Export to Markdown — export conversation with sources for archiving
- Embedding cache — avoid re-embedding unchanged documents

RAG Chat API Example

```
curl -X POST http://localhost:9000/api/rag/chat \
-H "Content-Type: application/json" \
-d '{
  "query": "What are the requirements for residency?",
  "collection": "estonian_laws",
  "model": "qwen3.6:27b",
  "language": "english"
}'
```

Architecture Details

Documents are split into chunks (default 512 tokens with 50-token overlap), embedded using bge-m3, and stored in Qdrant with metadata (source file, page number, chunk index). At query time, the user question is embedded and the top-K most similar chunks are retrieved (default K=5). These chunks are injected into the LLM prompt as context, and the model generates an answer with citations.

7. Telegram AI Bot

The Telegram module turns your Telegram account into an AI-powered conversational agent using Telethon (User API). It responds from your own account with configurable personas, supporting text, voice messages (with voice cloning), and photo analysis.

14 Built-in Personas

N	Persona	Style / Description
#1	Philosopher	Existential sage mixing Nietzsche with memes
#2	Street Philosopher	Street-smart guy with unexpectedly deep thoughts
#3	IT Demon	Perceives reality as code, people as processes
#4	Granny from 2077	Caring grandma from a cyberpunk future
#5	Noir Detective	Hard-boiled 40s detective in modern reality
#6	Nerd Pirate	Pirate sailing the internet seeking knowledge
#7	Cat Overlord	Arrogant cat, considers humans as servants
#8	Conspiracy Nut	Sees absurd conspiracies everywhere (+ voice mode)
#9	Budget Shakespeare	Pompous theatrical language about mundane things
#10	Polite Zombie	Craves brains but discusses it with manners
#11	Corporate Bot	Parody manager: everything is KPIs and synergy
#12	Meme Capybara	Maximum zen and chill (attaches capybara photos)
#13	Crypto Maniac	Deranged crypto investor, emotional rollercoaster
#14	Custom Character	User-defined personality and system prompt

Voice Clone Pipeline

```
Incoming voice msg
|-> ffmpeg (decode OGG to WAV)
|-> Whisper STT (speech to text, auto language detect)
|-> LLM (generate response in active persona's style)
|-> Qwen3-TTS (clone sender's voice from 3-second sample)
|-> ffmpeg (encode WAV to OGG Opus)
|-> Send voice reply in cloned voice
```

Vision Pipeline

```
Incoming photo
|-> Auto-unload current Ollama models (free VRAM)
|-> Load MiniCPM-V 8B (vision model)
|-> Analyze image + generate response in persona style
|-> Unload vision model, restore previous model
|-> Send text reply
```

Features

- Automatic language detection — responds in the sender's language
- 5-message conversation memory per chat for contextual replies
- Cooldown system (30s default) to prevent spam and rate limiting

- Session logging for debugging, review, and conversation export
- CRUD for custom personas via panel UI — create unlimited characters
- Per-persona voice reply toggle — some personas voice-only by default
- Capybara mode — attaches random capybara photo to every reply

8. LoRA Fine-Tuning

The LoRA tab provides a web interface for fine-tuning language models using Unsloth, which delivers 2x training speed and 60% less memory usage compared to standard PyTorch training. Adapters can be exported and used directly with Ollama.

16 Supported Base Models

Model	Param:	VRAM	Train Time*
unsloth/Nemotron-Mini-4B-Instruct	4B	~5 GB	~15 min/epoch
unsloth/Llama-3.1-8B-Instruct	8B	~8 GB	~25 min/epoch
unsloth/Llama-3.2-3B-Instruct	3B	~4 GB	~10 min/epoch
unsloth/Llama-3.2-1B-Instruct	1B	~2 GB	~5 min/epoch
unsloth/Qwen2.5-7B-Instruct	7B	~7 GB	~20 min/epoch
unsloth/Qwen2.5-32B-Instruct	32B	~22 GB	~90 min/epoch
unsloth/gemma-2-9b-it	9B	~9 GB	~28 min/epoch
unsloth/gemma-2-2b-it	2B	~3 GB	~8 min/epoch
unsloth/Mistral-7B-Instruct-v0.3	7B	~7 GB	~20 min/epoch
unsloth/Phi-3.5-mini-instruct	3.8B	~5 GB	~12 min/epoch
unsloth/Phi-4	14B	~12 GB	~45 min/epoch
unsloth/DeepSeek-R1-Distill-Qwen-7B	7B	~7 GB	~22 min/epoch
unsloth/DeepSeek-R1-Distill-Qwen-14B	14B	~12 GB	~48 min/epoch
unsloth/Llama-3.3-70B-Instruct	70B	~22 GB	~180 min (4bit)
unsloth/SmolLM2-1.7B-Instruct	1.7B	~2.5 GB	~7 min/epoch
unsloth/Llama-3.1-8B	8B	~8 GB	~25 min/epoch

* Training times are approximate for 1000-sample datasets on RTX 3090. Actual time depends on dataset size, sequence length, and batch size.

Training Parameters

- Rank (r):** LoRA rank dimension (4-128, default 16)
- Alpha:** Scaling factor (8-256, default 32)
- Epochs:** Training iterations (1-50)
- Batch Size:** Samples per gradient step
- Learning Rate:** Optimizer step size (e.g. 2e-4)
- Max Seq Length:** Context window for training samples (512-32768)

Workflow

1. Select base model from the 16 available options.
2. Upload training data in JSONL format (instruction/input/output fields).
3. Configure training hyperparameters.
4. Start training — output streams live to the UI.
5. Monitor loss curves in real-time.
6. Stop early if convergence reached.
7. Export LoRA adapter for use with Ollama.

Dataset Format

Training data must be in JSONL format with the following structure:

```
{"instruction": "Translate to French", "input": "Hello", "output": "Bonjour"}  
{"instruction": "Summarize", "input": "Long text...", "output": "Summary..."}  
{"instruction": "Fix the bug", "input": "def f(x):\n  return x+", "output": "def f(x):\n  return x + 1"}
```

Output & Export

After training completes, the LoRA adapter is saved to the output directory. You can merge it with the base model and create an Ollama-compatible GGUF file. The adapter typically adds only 50-200 MB on top of the base model, making it efficient to store multiple fine-tuned variants.

LoRA adapters are small (50-200 MB) and can be swapped without reloading the full base model. Train multiple adapters for different tasks.

9. Generation Pipeline

NeuralForge provides an integrated pipeline for generating images, videos, and 3D models. The system automatically manages VRAM between pipeline stages, stopping one heavy service before starting the next.

Pipeline Stages

Step	Engine	VRAM	Description
1. Image	ComfyUI + FLUX Klein	8-22 GB	Text-to-image, various styles
2. Video	Wan2GP + Wan 2.2	12-24 GB	Image-to-video or text-to-video
3. 3D	Hunyuan3D v2	13-20 GB	Image-to-3D mesh generation

Full pipeline runs sequentially on a single GPU. Each stage auto-stops the previous heavy_gpu service before starting.

Automation Level

The pipeline is fully automated: provide a text prompt and it generates an image, animates it into a video, then creates a 3D model. Each stage can also be run independently via API calls.

5 Built-in Examples

- Robot:** Futuristic humanoid robot in a neon-lit cityscape
- Dragon:** Fantasy dragon with detailed iridescent scales
- Car:** Concept sports car with aerodynamic design
- Cat:** Photorealistic Persian cat portrait in studio lighting
- Sword:** Ornate fantasy sword with glowing rune engravings

API Usage Example

```
curl -X POST http://localhost:9000/api/smm/generate-image \  
-H "Content-Type: application/json" \  
-d '{"prompt": "a futuristic robot in a neon city"}'
```

10. SMM AI Department

The SMM module is a complete AI-powered social media management system. It covers the full content lifecycle: trend discovery, AI-generated posts with images, scheduling, multi-platform publishing, and engagement analytics. The system supports brand profiles with custom voice, tone, and target audience settings.

Workflow Overview

```
Trend Scanning (6 sources)
|-> Topic Selection
|-> AI Post Generation (2-pass: research + brand voice)
|-> Image Generation (FLUX Klein via ComfyUI)
|-> Content Queue (edit, schedule, reorder)
|-> Multi-Platform Publish (7 platforms)
|-> Analytics Dashboard (engagement tracking)
```

7 Supported Platforms

Platform	Auth	Capabilities
Telegram	Bot API	Text + image, channel/group posting
Twitter/X	OAuth 2.0	Tweet + media, thread support
LinkedIn	OAuth 2.0	Article + image, company pages
Facebook	Graph API	Post + image, page publishing
Instagram	Graph API	Image + caption, carousel support
Threads	Graph API	Text + image posting
Bluesky	AT Proto	Text + image, alt text support

Key Features

- 6 trend sources: Google Trends, Reddit, HackerNews, GitHub Trending, arXiv, custom RSS
- 2-pass post generation: first draft with research context, then polish with brand voice
- AI image generation: auto-generate visuals matching post content via FLUX Klein
- Batch generation: create multiple posts in one operation for content calendaring
- Content queue with drag-and-drop reordering and scheduled auto-publishing
- Calendar view: visual timeline of scheduled and published content
- Token health monitoring: track expiration and validity of all platform tokens
- Per-post analytics: engagement metrics, reach, and click tracking
- GitHub repository search: find trending repos for tech content
- Profile management: multiple brand profiles with independent settings

Content Generation Workflow

Post generation uses a 2-pass approach: First, the AI researches the topic using trend data, web search results, and profile context to create a draft. Then, a second pass applies the brand's voice, tone, and formatting rules to produce the final post. Each post can be customized for the target platform (different length, hashtags, format).

Auto-Scheduler

The content queue supports scheduled publishing with configurable time slots. The calendar view shows a visual

timeline of past, scheduled, and draft posts. Posts can be rescheduled via drag-and-drop. The auto-scheduler distributes batch-generated content across optimal posting times.

API Usage Example

```
curl -X POST http://localhost:9000/api/smm/generate \  
-H "Content-Type: application/json" \  
-d '{  
  "profile_id": "neural_overlord",  
  "topic": "Latest advances in local LLMs",  
  "platforms": ["telegram", "twitter", "linkedin"]  
}'
```

11. MCP Server

NeuralForge exposes 24 tools via the Model Context Protocol (MCP) for integration with Claude Code, Claude Desktop, and other MCP-compatible clients. The MCP server communicates with NeuralForge via its REST API.

All 24 MCP Tools

Tool	Description
get_system_status	Hardware metrics: GPU, RAM, CPU, disk, services
start_service	Start a specific NeuralForge service by name
stop_service	Stop a specific NeuralForge service by name
rag_search	Semantic search across RAG collections
rag_list_collections	List all Qdrant vector collections
rag_index_file	Index a single file into a collection
rag_index_directory	Recursively index a directory
ask_rag	Full RAG chat: retrieve + generate answer
get_storage_info	Disk usage breakdown by service
cleanup_storage	Clean logs/cache for a service
finetune_status	Current LoRA training status and progress
finetune_start	Start a LoRA fine-tuning run
finetune_stop	Stop running LoRA training
run_pipeline	Trigger image/video/3D generation pipeline
run_backup	Create platform backup archive
get_gpu_processes	List GPU processes with VRAM usage
ollama_loaded_models	Currently loaded Ollama models
convert_audio	Convert audio between formats
run_agent	Run a solo agent task
run_agent_team	Run a team chain of agents
run_orchestrator	Run the AI orchestrator
stop_all_and_free_vram	Stop heavy services + unload models
generate_image	Generate image via ComfyUI + FLUX
check_health	Health check with alert status

MCP Configuration

Add to your Claude Code .mcp.json or claude_desktop_config.json:

```
{
  "mcpServers": {
    "neuralforge": {
      "command": "python",
      "args": ["mcp_server.py"],
      "cwd": "/home/user/ai-panel"
    }
  }
}
```

MCP tools allow Claude Code to directly control your NeuralForge instance: start services, run agents, search RAG, and more.

MCP Resource

In addition to tools, the MCP server exposes a `status://system` resource that provides a read-only snapshot of the entire system state including all service statuses, GPU metrics, loaded models, and active tasks. MCP clients can subscribe to this resource for real-time system awareness.

Usage Example with Claude Code

Once configured, you can ask Claude Code to interact with NeuralForge directly:

```
> "Start Ollama and check what models are loaded"
-> Claude calls: start_service("ollama")
-> Claude calls: ollama_loaded_models()
-> Returns list of loaded models with VRAM usage

> "Index my project docs into RAG and ask about the API"
-> Claude calls: rag_index_directory("/path/to/docs")
-> Claude calls: ask_rag("How does the API work?")
-> Returns answer with source citations
```

12. Installation & Startup

Prerequisites

- Ubuntu 22.04+ or compatible Linux distribution
- NVIDIA GPU with 12+ GB VRAM (24 GB recommended)
- Python 3.10+ with pip and venv
- Docker and Docker Compose v2
- NVIDIA Container Toolkit (for GPU access in Docker containers)
- ffmpeg (for audio processing)

Install Process

```
git clone https://github.com/DefinitelyN0tMe/neuralforge.git
cd neuralforge
chmod +x install.sh
./install.sh
```

The installer: 1) Checks for Python, GPU, Docker, ffmpeg and Ollama (it does not install them). 2) Creates venv/ and installs requirements.txt. 3) Patches hardcoded paths for your install directory and home. 4) Creates telegram_config.json from the example. 5) Optionally installs a systemd USER unit (~/.config/systemd/user/ai-panel.service) for auto-start. Do not also create a system unit in /etc/systemd/system: two units fight over port 9000.

systemd User Service

```
# ~/.config/systemd/user/ai-panel.service
[Unit]
Description=NeuralForge
After=network.target

[Service]
Type=simple
WorkingDirectory=/home/user/neuralforge
ExecStart=/home/user/neuralforge/venv/bin/python3 -u server.py
Restart=on-failure
RestartSec=5
Environment=DOCKER_HOST=unix:///var/run/docker.sock

[Install]
WantedBy=default.target

# Keep it running without an open login session:
sudo loginctl enable-linger $USER
```

Server Management

Control commands:

```
# Start
systemctl --user start ai-panel

# Stop
systemctl --user stop ai-panel

# Restart (from API - preferred)
curl -X POST http://localhost:9000/api/restart

# View logs
journalctl --user -u ai-panel -f
```

Always prefer POST /api/restart over killing the server process. The API restart preserves state and performs clean shutdown.

13. API Reference (84 Endpoints)

NeuralForge exposes 84 REST API endpoints + 1 WebSocket at localhost:9000. All endpoints accept and return JSON. Below is the complete list organized by category, extracted directly from server.py and smm/routes.py.

System & Actions

Method	Endpoint	Description
GET	/	Web UI (HTML)
GET	/api/health	Health check with alerts
GET	/api/status	All module statuses
GET	/api/storage	Disk usage by category
POST	/api/restart	Restart NeuralForge server
POST	/api/actions/stop-all-heavy	Stop all heavy GPU services
POST	/api/actions/start-basics	Start Ollama + Qdrant + WebUI
POST	/api/actions/free-vram	Unload all Ollama models
GET	/api/llm-models	Installed Ollama chat models (live)

Metrics

Method	Endpoint	Description
GET	/api/metrics/summary	LLM calls per model/source (?hours=24)
GET	/api/metrics/timeseries	GPU util/VRAM/temp samples over time
GET	/api/metrics/services	Service uptime % per module
GET	/api/metrics/recent	Latest LLM calls (?limit=50)

Module Lifecycle

Method	Endpoint	Description
POST	/api/module/{filename}/start	Start a module
POST	/api/module/{filename}/stop	Stop a module
GET	/api/module/{filename}/log	Last 50 log lines
POST	/api/cleanup/{module_file}	Clean module storage

Secrets

Method	Endpoint	Description
GET	/api/secrets	List stored API keys
POST	/api/secrets	Save / update API key

Telegram Bot

Method	Endpoint	Description
GET	/api/telegram	Bot config + session list
GET	/api/telegram/auth/status	Telegram account login state
POST	/api/telegram/auth/send-code	Send login code to phone
POST	/api/telegram/auth/sign-in	Sign in with code (+2FA)
GET	/api/telegram/session/{id}	Single session details
DELETE	/api/telegram/session/{id}	Delete a session

POST	/api/telegram/config	Update bot config
POST	/api/telegram/personas	Create custom persona
PUT	/api/telegram/personas/{id}	Update persona
DELETE	/api/telegram/personas/{id}	Delete persona
POST	/api/telegram/start	Start bot
POST	/api/telegram/stop	Stop bot
DELETE	/api/telegram/sessions	Clear all sessions
DELETE	/api/telegram/messages	Clear message history

Agents

Method	Endpoint	Description
GET	/api/agents	Roles, tools, models, status
POST	/api/agents/run	Run solo agent task
POST	/api/agents/upload	Upload file for agent
GET	/api/agents/pdf/{filename}	Serve agent-generated PDF
POST	/api/agents/run-team	Run team chain
POST	/api/agents/run-orchestrator	Run AI orchestrator
GET	/api/agents/status	Running agents status
POST	/api/agents/stop	Stop running agent
GET	/api/agents/history	List past tasks
GET	/api/agents/history/{filename}	Read history file
DELETE	/api/agents/history/{filename}	Delete history file
DELETE	/api/agents/history	Clear all history

RAG

Method	Endpoint	Description
GET	/api/rag/status	Collections + stats
POST	/api/rag/index	Index directory into Qdrant
POST	/api/rag/upload-and-index	Upload file & index
DELETE	/api/rag/collection/{name}	Delete collection
POST	/api/rag/chat	RAG-augmented chat

LoRA Fine-Tuning

Method	Endpoint	Description
GET	/api/finetune	Models, status, datasets
POST	/api/finetune/start	Start training run
POST	/api/finetune/stop	Stop training
POST	/api/finetune/upload-dataset	Upload JSONL dataset

SMM — Profiles

Method	Endpoint	Description
GET	/api/smm/profiles	List social profiles
GET	/api/smm/profiles/{id}	Single profile details
POST	/api/smm/profiles	Create profile
PUT	/api/smm/profiles/{id}	Update profile
DELETE	/api/smm/profiles/{id}	Delete profile

SMM — Content

Method	Endpoint	Description
POST	/api/smm/generate	Generate post text
POST	/api/smm/generate-image-prompt	Generate image prompt
POST	/api/smm/generate-image	Generate post image
POST	/api/smm/regen-post	Regenerate existing post
POST	/api/smm/batch-generate	Batch generate posts
GET	/api/smm/batch-status	Batch job status
POST	/api/smm/publish	Publish to platform

SMM — Queue & Calendar

Method	Endpoint	Description
GET	/api/smm/queue	Content queue list
POST	/api/smm/queue	Add to queue
PUT	/api/smm/queue/{id}	Update queue item
DELETE	/api/smm/queue/{id}	Remove from queue
GET	/api/smm/calendar	Calendar view data
GET	/api/smm/events	Scheduled events

SMM — Trends & Analytics

Method	Endpoint	Description
POST	/api/smm/trends/scan	Scan trend sources
GET	/api/smm/trends/history	Trend scan history
GET	/api/smm/trends/by-id	Trend by ID
GET	/api/smm/trends/latest	Latest trends
GET	/api/smm/analytics	Analytics overview
GET	/api/smm/analytics/post/{id}	Per-post analytics
GET	/api/smm/token-health	Token health status
POST	/api/smm/token-refresh	Refresh expired token

SMM — Storage & Search

Method	Endpoint	Description
GET	/api/smm/storage	SMM storage usage
POST	/api/smm/cleanup	Clean SMM data
POST	/api/smm/github-search	Search GitHub repos
GET	/api/smm/image/{filename}	Serve generated image

WebSocket

Method	Endpoint	Description
WS	/ws	Real-time events stream

14. Hardware Requirements

Hardware Specifications

Component	Minimum	Recommended	Tested On
GPU VRAM	12 GB	24 GB	RTX 3090 24GB
RAM	16 GB	64+ GB	128 GB DDR4 ECC
Disk	50 GB free	200+ GB SSD	2 TB NVMe RAID
CPU	4+ cores	8+ cores	Threadripper PRO
Network	Localhost	Gigabit	10 GbE

Software Requirements

- OS:** Ubuntu 22.04+ (tested on Ubuntu 24.04)
- Python:** 3.10+ (3.12 recommended)
- Docker:** 20.10+ with Docker Compose v2
- NVIDIA Driver:** 535+ with CUDA 12.x
- NVIDIA Container Toolkit:** Required for GPU access in Docker
- ffmpeg:** Required for audio processing (voice clone, STT)

VRAM Usage Patterns

Not all services run simultaneously. The heavy_gpu group ensures only one large service uses the GPU at a time. Typical usage patterns on a 24 GB GPU:

Workload	Services Used	VRAM
Chat + RAG	Ollama (4–16 GB) + Qdrant	8–18 GB
Image Generation	ComfyUI + FLUX Klein	8–22 GB
Video Generation	Wan2GP + Wan 2.2	12–24 GB
3D Generation	Hunyuan3D v2	13–20 GB
Telegram Bot (full)	Ollama + Whisper + TTS + Vision	12–18 GB
LoRA Training	Unsloth + base model	5–22 GB
Agent Workflow	Ollama + tools	6–20 GB

A 24 GB GPU (RTX 3090/4090) can run most features. 12 GB GPUs work with smaller models (9B) and careful VRAM management. The heavy_gpu mutex system prevents OOM crashes automatically.

Port Allocation

Port	Service	Description
9000	NeuralForge	Main web UI and API server
11434	Ollama	LLM inference API
8188	ComfyUI	Image generation interface
7860	Wan2GP	Video generation
7870	Hunyuan3D	3D model generation

7880	ACE-Step	Music generation
7890	Qwen3-TTS	Text-to-speech / voice clone
7895	Whisper	Speech-to-text
6333	Qdrant	Vector database
8080	Open WebUI	Chat interface
3000	Perplexica	AI search
8888	SearXNG	Meta search engine

Disk Space Breakdown

Approximate disk usage by component:

Component	Size	Notes
Ollama models	5-80 GB	Depends on model count and sizes
ComfyUI + checkpoints	10-30 GB	FLUX Klein checkpoint + VAE
Wan2GP model	15-25 GB	Wan 2.2 video model weights
Hunyuan3D model	10-20 GB	3D generation weights
Docker images	5-10 GB	Qdrant, WebUI, Perplexica, SearXNG
Python environments	3-8 GB	Virtual environments + packages
User data	Variable	RAG collections, LoRA datasets, logs